WebSocket-Native Agentic Coding Harness: A Deep-Dive Architectural Blueprint

Executive Summary

This report explores the feasibility and architecture of a WebSocket-native agentic coding tool that combines the best architectural elements of Anthropic's Claude Code harness with OpenAI's Responses API WebSocket mode — eliminating HTTP request overhead and proxy layers entirely. The analysis draws from the Claude Code source leak (512,000 lines of TypeScript across 1,900 files), OpenAI's official Responses API and WebSocket documentation, the OpenAI Agents SDK, the Codex CLI open-source harness, and MCP protocol specifications.

The core finding: Claude Code's agentic loop architecture is fundamentally transport-agnostic. The harness logic — tool execution, permission gating, self-healing, multi-agent orchestration, and context management — is pure application-layer code. Only the API call layer and context compaction subsystem are coupled to HTTP, and both have direct WebSocket-native replacements in the Responses API. A WebSocket-native replica is not just possible — it is architecturally superior, because the 5-tier compaction system that Claude Code engineers spent enormous effort building exists primarily to compensate for HTTP's statelessness. WebSocket mode's server-side `previous_response_id` chaining and connection-local in-memory cache eliminate most of that complexity entirely.

Part 1: Claude Code Harness Architecture — What Makes It Good

The Agentic Loop (query.ts)

The heart of Claude Code is a 1,729-line `while(true)` async generator loop in `query.ts`, orchestrating the entire cycle: receiving user input, calling the model, parsing responses, executing tools, and feeding results back (Bits, Bytes and Neural Networks). This is wrapped by a 1,295-line outer loop in `QueryEngine.ts` that handles retries, budget enforcement, permission checks, and max turn limits (Victor Antos).

The exact internal sequence, reconstructed from the leaked source:

Step	What Happens	Transport Dependency
1	Assemble context (system prompt + CLAUDE.md + memory + conversation history)	None — pure harness logic
2	Pre-request compaction (5-stage pipeline)	HTTP-coupled — exists because full context must be re-sent
3	Call model API (streaming async generator)	HTTP-coupled — currently sends Anthropic `/v1/messages` POST

Step	What Happens	Transport Dependency
4	Parse response (text blocks + tool_use blocks)	None — pure parsing
5	Permission gate (deny → allow → ask user)	None — pure harness logic
6	Execute tools (reads parallel, writes serial)	None — local execution
7	Feed results back (tool results appended to conversation)	None — pure harness logic
8	Context check (too large? compact → loop)	Partially HTTP-coupled
9	Termination (no tool calls = done; or budget exceeded)	None — pure harness logic

Steps 4–7 and 9 are completely transport-agnostic. Only Steps 2, 3, and 8 touch the wire protocol (Penligent).

The StreamingToolExecutor — Parallel Tool Execution During Generation

One of Claude Code's most impactful latency optimizations: the `StreamingToolExecutor` begins executing tools as they arrive in the API stream, not after the full response completes. Tools marked `isConcurrencySafe` run in parallel; non-concurrent tools get exclusive access. Results are buffered and emitted in request order (Victor Antos). Large tool results are persisted to disk — the conversation holds a file reference, not the raw content, preventing memory bloat in long-running sessions.

This is pure harness logic. It needs streaming events from the model, but those events can arrive over WebSocket identically to how they arrive over SSE. The `StreamingToolExecutor` pattern transfers directly to a WebSocket-native implementation.

The Tool System — 80+ Permission-Gated Modules

Every tool defines its own input schema (Zod v4), permission level, and execution logic independently. There is no shared mutable state between tools (WaveSpeed). The tool interface is minimal:

```
interface Tool {
  name: string
  schema: z.ZodSchema      // Zod input validation
  description: string
  execute(input, context: ToolUseContext): Promise<ToolResult>
  canUseInNonInteractive?: boolean
  isConcurrencySafe?: boolean
}
```

`AgentTool` is the architecturally significant one — it spawns sub-agents as just another tool call, no special orchestration layer required. Sub-agents are first-class citizens of the same tool registry (WaveSpeed). This keeps the architecture flat and predictable.

The permission system is three-tiered:

1. **Tier 1 — Rules-based fast path:** Pattern matching on tool inputs using glob and regex (62KB of TypeScript in `filesystem.ts`)
2. **Tier 2 — ML classifier:** Calls the Claude API to classify whether a bash command is dangerous (feature-gated as `BASH_CLASSIFIER`)
3. **Tier 3 — User prompts:** Interactive approval dialogs with plan mode preview

All of this is transport-agnostic. The permission system is pure harness logic (Victor Antos).

Multi-Agent Orchestration — The Mailbox Pattern

Claude Code implements a coordinator/worker split with a mailbox pattern for dangerous operations. A worker agent executing a task cannot independently approve a high-risk operation. Instead, it sends a request to the coordinator's mailbox and waits. The coordinator evaluates and either approves or rejects. An atomic claim mechanism prevents two workers from handling the same approval simultaneously (WaveSpeed).

Shared memory space across all agents means the team maintains coherent context without redundant re-fetching. A key insight from the Latent Space analysis: Claude Code uses the KV cache to create a fork-join model for subagents, meaning parallelism is basically free — subagents contain the full parent context and do not have to repeat work.

Three isolation modes for `AgentTool`:

1. **In-process** — shared memory, same context
2. **Git worktree** — temporary `git worktree` gives child agent a sandboxed repo copy
3. **Remote (CCR)** — cloud execution via Cross-Code Runtime

All of this is harness-level logic that is completely independent of whether the model API call uses HTTP or WebSocket (Victor Antos).

Self-Healing and Resilience

The self-healing query loop triggers compression automatically when the context budget approaches its limit, carving out a buffer before the ceiling and generating a structured summary. A circuit breaker halts after three consecutive compression failures — no infinite loops (WaveSpeed).

Diminishing returns detection is built in: if Claude continues 3 consecutive times but produces fewer than 500 tokens each time, the system decides "continuing further is pointless" and stops. It keeps going until 90% of the budget is consumed but will not spin its wheels (Bits, Bytes and Neural Networks).

Stop hooks execute user-defined validation logic. A hook saying "don't stop unless tests pass" runs when Claude tries to finish. If it fails, the hook's error message is injected into the conversation and Claude tries again.

The 5-Stage Compaction Pipeline — The HTTP Tax

This is the most architecturally significant subsystem for understanding why a WebSocket-native approach is superior. Claude Code runs five compaction mechanisms in sequence before every API call (Bits, Bytes and Neural Networks):

Stage	Mechanism	What It Does	API Cost
1	Tool Result Budget	Caps oversized tool results to use context window efficiently	None
2	Snip Compact	Cheapest option — trims history when `HISTORY_SNIP` flag is active	None
3	MicroCompact	Cache-aware tool result clearing — edits cached content locally	None
4	Context Collapse	Staged reduction of conversation segments	Low
5	Auto-Compact	Full summarization when threshold exceeded — reserves 13K-token buffer, generates up to 20K-token summary	High

Post Auto-Compact, the full compact mechanism compresses the entire conversation, then re-injects recently accessed files (capped at 5,000 tokens per file), active plans, and relevant skill schemas. Working budget resets to 50,000 tokens (WaveSpeed).

This entire 5-stage pipeline exists because HTTP is stateless. Every HTTP request must carry the full conversation history. The compaction system is the engineering cost of that constraint. In a WebSocket-native architecture with `previous_response_id` chaining, the server retains conversation state in a connection-local in-memory cache. The client sends only incremental inputs — new tool outputs and the next user message. The majority of this compaction pipeline becomes unnecessary overhead.

Memory Architecture — CLAUDE.md + Auto Memory

Two cross-session memory systems work in parallel:

System	Who Writes It	Scope	Purpose
CLAUDE.md	Human	Project, user, or org	Stable rules, architecture notes, workflows
Auto Memory	Claude	Per working tree	Learned build commands, debugging patterns, preferences

CLAUDE.md files are loaded hierarchically — Claude Code walks up the directory tree from the current working directory, loading applicable files. In monorepos, this makes CLAUDE.md closer to repo-local policy than a personal note file (Penligent).

An unreleased `KAIROS_DREAM` feature flag reveals nightly memory distillation — merging memories, deduplication, pruning, and contradiction removal (Victor Antos).

Hooks — Runtime Policy Control

Hooks are user-defined shell commands, HTTP endpoints, LLM prompts, or agent hooks that run at specific lifecycle points. The most critical is `PreToolUse` — it fires after Claude creates tool parameters and before the tool call executes. It can allow, deny, ask, or defer the tool call, modify tool input, and append additional context. Multiple hooks can disagree and precedence rules resolve conflicts (Penligent).

Telemetry — Behavioral Signals

Two telemetry signals worth noting: a **frustration metric** tracking swearing frequency as a UX signal, and a **"continue" counter** tracking how often users type "continue" mid-session — a proxy for stalls where the agent lost momentum (WaveSpeed).

Unreleased Feature Flags

108 feature-gated modules stripped from external builds. Notable gates from the leaked source (Victor Antos):

Gate	Purpose
`KAIROS`	Long-running persistent background agent mode
`COORDINATOR_MODE`	Multi-worker agent distribution
`VOICE_MODE`	Speech input with hold-to-talk
`PROACTIVE`	Autonomous agent behavior
`ULTRAPLAN`	Browser-based plan approval with teleport back to terminal
`KAIROS_DREAM`	Nightly memory distillation
`BYOC_ENVIRONMENT_RUNNER`	Bring-your-own-compute

Part 2: OpenAI Responses API WebSocket Mode — The Transport Layer

Core Mechanics

The Responses API supports a WebSocket mode at `wss://api.openai.com/v1/responses` designed explicitly for "long-running, tool-call-heavy workflows" — OpenAI's own documentation cites "agentic coding or orchestration loops with repeated tool calls" as the primary use case (OpenAI WebSocket Mode).

Key properties:

Property	Detail
Connection	Persistent WebSocket to `wss://api.openai.com/v1/responses`
Auth	`Authorization: Bearer {API_KEY}` header on connect

Property	Detail
Turn initiation	Client sends `response.create` JSON frame
Continuation	`previous_response_id` + only new input items (incremental)
State retention	Connection-local in-memory cache of most recent response
Latency gain	Up to ~40% faster end-to-end on workflows with 20+ tool calls
Connection limit	60 minutes; reconnect and chain with `previous_response_id`
Parallelism	One in-flight response per connection; use multiple connections for parallel runs
ZDR compatible	Works with `store=false` and Zero Data Retention

Incremental Input — The Key Innovation

On continuation turns, the client sends only:

- `previous_response_id` pointing to the prior response
- `input` containing only new items (tool outputs, next user message)

The server reconstructs full context from its in-memory cache. This eliminates the need to re-send the entire conversation history on every turn — which is exactly the problem Claude Code's 5-stage compaction pipeline was built to manage (OpenAI WebSocket Mode).

Warmup Requests

Clients can send `response.create` with `generate: false` to warm up request state — pre-loading tools, instructions, and custom messages for an upcoming turn. This returns a response ID that can be chained from with `previous_response_id`, giving the next generated turn a head start (OpenAI WebSocket Mode).

Server-Side Compaction (context_management)

When enabled, the server automatically compacts context when token count crosses a configured `compact_threshold`. The response stream includes an encrypted compaction item that carries forward key prior state using fewer tokens. This is fully ZDR-friendly with `store=false` (OpenAI Compaction Guide).

```
response = client.responses.create(
    model="gpt-5.3-codex",
    input=conversation,
    store=False,
    context_management=[{"type": "compaction", "compact_threshold": 200000}],
)
```

This replaces Claude Code's entire client-side compaction pipeline with a single server-side parameter. The server handles when and how to compact, emitting an opaque compaction item in the stream that the client just passes through on subsequent turns.

Standalone Compact Endpoint

For explicit control, `/responses/compact` is a stateless endpoint that accepts a full context window and returns a compacted one. This is useful at WebSocket reconnection boundaries — compact the context, then start a new chain on the fresh socket (OpenAI Compaction Guide).

Reconnection and Recovery

Three patterns for handling connection closure:

1. **Persisted state** (`store=true`): Continue with `previous_response_id` and new input items on a new socket
2. **ZDR mode** (`store=false`): If in-memory cache is lost, start a new response with full input context
3. **Compacted**: Use `/responses/compact` output as the base `input` for the new response, then append latest items

Part 3: The OpenAI Agents SDK — Production Validation

The OpenAI Agents SDK (Python) provides a production-validated implementation of the exact pattern being proposed here. The `responses_websocket_session()` async context manager creates a shared WebSocket session that keeps connections warm across turns and nested agent-as-tool runs (OpenAI Agents SDK).

```
async with responses_websocket_session(api_key="...") as session:
    result = await session.run(
        starting_agent=coding_agent,
        input="Refactor the auth module to use JWT"
    )
```

Key design decisions in the SDK:

- **Shared** `RunConfig`: One WebSocket-capable `MultiProvider` is injected into every `Runner.run()` call within the session
- **Prefix-based model routing**: Supports `openai/gpt-5.4` style routing while keeping WebSocket connections warm
- **Nested agent inheritance**: Sub-agents spawned as tools inherit the same `run_config`, meaning they share the WebSocket connection without additional infrastructure
- `OpenAIResponsesWSModel`: A full implementation of the Model interface over WebSocket transport, with retry logic including `ModelRetryAdvice` for connection and timeout failures

The JavaScript SDK offers `withResponsesWebSocketSession()` with identical semantics — runs a callback within a session-scoped WebSocket provider and closes it afterwards (OpenAI Agents JS SDK).

This is the production proof that the architecture works end-to-end for agentic coding workflows over WebSocket with zero intermediary layers.

Part 4: Codex CLI Harness — The Other Reference Architecture

OpenAI's Codex CLI (open-source, written in Rust) provides a second reference implementation of a production agentic coding harness. Key architectural parallels and contrasts with Claude Code (ZenML Analysis):

Shared Patterns

Pattern	Claude Code	Codex CLI
Core loop	`while(true)` async generator in `query.ts`	State machine agent loop in Rust
Tool execution	StreamingToolExecutor (parallel during generation)	Shell/file tools in sandbox
Permission system	3-tier (rules → ML classifier → user prompt)	Approval modes (suggest, auto-edit, full-auto)
Context management	5-stage client-side compaction	Auto-compact via `/responses/compact`
Instruction hierarchy	CLAUDE.md (directory walk)	AGENTS.md (directory walk, 32KB limit)
Subagents	AgentTool (in-process, worktree, remote)	Agent-as-tool in Agents SDK
Session persistence	Checkpoint/restore with transcripts	Thread lifecycle with event history
MCP integration	First-class MCP client (stdio, SSE, WebSocket)	MCP server mode + MCP tool support

Key Differences

Aspect	Claude Code	Codex CLI
Language	TypeScript (Bun runtime)	Rust
Transport	Anthropic HTTP `/v1/messages`	Responses API HTTP (deliberately avoids `previous_response_id` for ZDR)
WebSocket	Not supported natively	Not used by CLI, but Agents SDK has full WebSocket session support
Cache strategy	Server-side KV cache with fork-join for subagents	Prefix preservation for prompt cache hits (linear vs. quadratic)
UI	Custom React-based terminal renderer (not npm ink — from-scratch Yoga flexbox port)	TUI via Rust
App Server	N/A	Bidirectional JSON-RPC over stdio — stable protocol for IDEs, web, desktop (OpenAI)

The Codex App Server Pattern

OpenAI's approach to exposing the harness is instructive for a WebSocket-native design. The Codex App Server is a bidirectional JSON-RPC API with three conversation primitives (OpenAI):

1. **Item**: Atomic unit of I/O with `started` → `delta` → `completed` lifecycle
2. **Turn**: One unit of agent work initiated by user input, containing a sequence of items
3. **Thread**: Durable container for an ongoing session, supporting create/resume/fork/archive

This Item/Turn/Thread hierarchy maps cleanly onto a WebSocket-native design — items stream as WebSocket frames, turns correspond to `response.create` sequences, and threads map to WebSocket connections or `previous_response_id` chains.

Part 5: MCP Protocol — The Extension Surface

The Model Context Protocol defines how external tools, data sources, and workflows connect to agentic systems. Claude Code is already a first-class MCP client supporting stdio, SSE, and WebSocket transports (Victor Antos).

Current MCP Transports

Transport	Use Case	Bidirectional
stdio	Local tools, same machine	Yes (stdin/stdout)
Streamable HTTP	Remote tools, web services	Partial (POST + SSE)
WebSocket (proposed)	Real-time bidirectional, session persistence	Full

MCP WebSocket Transport Proposal (SEP-1287)

An active specification proposal adds WebSocket as a native MCP transport (GitHub SEP-1288). Key design decisions:

- **Required session IDs** (vs. optional in Streamable HTTP) — because WebSocket implies bidirectional use
- **Single connection per session** — eliminates distributed state coordination
- **Mandatory message envelope** with `mcpSessionId` and `type: "mcp"` fields
- **Universal URL support** — same URL serves HTTP POST, SSE GET, and WebSocket upgrade
- **Backward compatible** — no changes to existing transports

This means a WebSocket-native harness could use WebSocket for both the model API connection (Responses API) and MCP tool connections, creating a fully WebSocket-native stack with no HTTP in the critical path.

Part 6: The Architecture — WebSocket-Native Hybrid Harness

Design Principles

Based on everything analyzed, the optimal architecture takes these elements:

From Claude Code (keep everything):

- The agentic loop structure (async generator, `while(true)`)
- StreamingToolExecutor (parallel tool execution during generation)
- 80+ tool system with per-tool permission gating and Zod schema validation
- Three-tier permission system (rules → classifier → user prompt)
- Multi-agent mailbox pattern with coordinator/worker split
- AgentTool with in-process, worktree, and remote isolation modes
- Self-healing loop with circuit breakers and diminishing-returns detection
- Hook system (PreToolUse, PostToolUse, PermissionRequest, ConfigChange)
- CLAUDE.md / AGENTS.md instruction hierarchy with directory walk
- Auto memory with per-worktree learned preferences
- Stop hooks for user-defined validation
- Frustration and "continue" telemetry signals
- Fork-join subagent model via KV cache sharing

From Claude Code (discard):

- Anthropic HTTP `/v1/messages` transport layer
- The entire 5-stage client-side compaction pipeline (replaced by server-side `context_management`)
- MicroCompact, AutoCompact, Full Compact client logic
- Snip compact and context collapse
- Client-side token counting and budget tracking for compaction decisions

From OpenAI (adopt):

- Responses API as the model interface
- WebSocket mode at `wss://api.openai.com/v1/responses`
- `previous_response_id` chaining with connection-local in-memory cache
- `response.create` with `generate: false` for warmup
- Server-side `context_management` with `compact_threshold`
- Standalone `/responses/compact` for reconnection boundaries
- `store=false` for ZDR compatibility
- Streaming event model (response.output_text.delta, response.output_item.added, etc.)

From Codex CLI (adopt):

- Item/Turn/Thread conversation primitives
- App Server JSON-RPC protocol pattern for IDE/web integration
- Prefix preservation discipline for prompt cache efficiency
- AGENTS.md cascading instruction precedence
- Skills as first-class instruction bundles

From MCP (adopt):

- WebSocket transport for MCP tool connections (when SEP-1287 ships)
- Tool namespace as governance vocabulary
- Dynamic tool discovery with deferred loading (tool names in context, full schemas on demand)

The Agentic Loop — WebSocket-Native Version

```
┌──────────────────────────────────────────────────────┐
│                   HARNESS CORE                       │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │           Agentic Loop (while true)          │    │
│  │                                              │    │
│  │  1. Assemble incremental input               │    │
│  │     (only NEW tool outputs + user message)   │    │
│  │                                              │    │
│  │  2. Send response.create over WebSocket      │──────────► wss://api.openai.com/v1/res
│  │     { previous_response_id, input: [...] }   │    │
│  │                                              │    │
│  │  3. Stream response events                   │◄──────  response.output_text.delta
│  │     (identical event model to SSE)           │◄──────  response.output_item.added
│  │                                              │    │
│  │  4. Parse response (text + tool_use blocks)  │    │
│  │                                              │    │
│  │  5. Permission gate                          │    │
│  │     ├─ Tier 1: Rules-based fast path         │    │
│  │     ├─ Tier 2: ML classifier                 │    │
│  │     └─ Tier 3: User approval                 │    │
│  │                                              │    │
│  │  6. StreamingToolExecutor                    │    │
│  │     ├─ Concurrent-safe tools: parallel       │    │
│  │     ├─ Non-concurrent tools: exclusive       │    │
│  │     └─ Results buffered in request order     │    │
│  │                                              │    │
│  │  7. Stop hooks / validation                  │    │
│  │     └─ If hook fails → inject error → loop   │    │
│  │                                              │    │
│  │  8. Termination check                        │    │
│  │     ├─ No tool calls → done                  │    │
│  │     ├─ Budget exceeded → done                │    │
│  │     └─ Diminishing returns → done            │    │
```

```
│  │                                                │    │
│  │  9. Loop back to step 1                        │    │
│  └────────────────────────────────────────────────┘    │
│                                                        │
│  ┌──────────────────────┐  ┌──────────────────────┐    │
│  │  Multi-Agent Layer   │  │   Hook System        │    │
│  │  Mailbox pattern     │  │   PreToolUse         │    │
│  │  Coordinator/Worker  │  │   PostToolUse        │    │
│  │  Fork-join subagents │  │   PermissionRequest  │    │
│  │  Worktree isolation  │  │   ConfigChange       │    │
│  └──────────────────────┘  └──────────────────────┘    │
│                                                        │
│  ┌──────────────────────┐  ┌──────────────────────┐    │
│  │  Memory Layer        │  │   MCP Client         │    │
│  │  AGENTS.md hierarchy │  │   stdio / WS / SSE   │    │
│  │  Auto memory         │  │   Dynamic discovery  │    │
│  │  Session persistence │  │   Deferred loading   │    │
│  └──────────────────────┘  └──────────────────────┘    │
└────────────────────────────────────────────────────────┘
```

What Changes vs. Claude Code

Claude Code (HTTP)	WebSocket-Native Harness	Impact
Full conversation sent every turn	Only incremental input sent per turn	Massive bandwidth reduction
5-stage client-side compaction pipeline	Server-side `context_management` with `compact_threshold`	Eliminates ~5,000+ lines of compaction code
Client tracks token budget for compaction decisions	Server handles compaction timing	Simpler client logic
MicroCompact (local cache edits, zero API calls)	Not needed — server retains full context	Client complexity removed
AutoCompact (13K buffer, 20K summary generation)	Replaced by server-side compact item in stream	Client complexity removed
Full Compact (conversation compression + file re-injection)	`/responses/compact` at reconnection boundaries only	Only needed at 60-min reconnect
`tool_result_budget` system to cap oversized outputs	Still useful as a local optimization	Keep — good engineering
Snip compact / Context Collapse	Not needed	Removed
Anthropic message format translation	Responses API native format	No translation layer
Proxy/bridge for multi-provider	`openai_prefix_mode` in Agents SDK handles routing	Native multi-model

Subagent Architecture Over WebSocket

For subagents, the WebSocket-native approach is strictly better:

- **One WebSocket connection per subagent** (OpenAI docs: "Use multiple connections if you need parallel runs")
- Each subagent gets its own `previous_response_id` chain
- The `responses_websocket_session()` pattern from the Agents SDK shows how sub-agents inherit the shared `run_config` and WebSocket provider
- Warmup requests (`generate: false`) can pre-load subagent context before the subagent starts generating

In Claude Code's current architecture, subagents leverage KV cache fork-join to avoid re-sending parent context. In the WebSocket-native version, `previous_response_id` chaining achieves the same effect — the server already has the parent context cached, and the subagent can chain from it.

Connection Lifecycle Management

```
Session Start:
  ws = connect("wss://api.openai.com/v1/responses")

  # Warmup: pre-load tools + instructions
  ws.send({ type: "response.create", generate: false,
            instructions: agents_md_content,
            tools: tool_definitions })
  warmup_response_id = receive_response_id()

First Turn:
  ws.send({ type: "response.create",
            previous_response_id: warmup_response_id,
            input: [user_message],
            context_management: [{type: "compaction", compact_threshold: 200000}] })

  # Stream events, execute tools, feed results back
  # All via response.create with incremental input

60-Minute Boundary:
  # Connection limit approaching
  compacted = http_post("/responses/compact", input=current_context)
  ws.close()

  ws = connect("wss://api.openai.com/v1/responses")
  ws.send({ type: "response.create",
            input: [...compacted.output, next_user_message] })
  # New chain starts; server-side compaction handles future growth

Reconnection (network drop):
  ws = connect("wss://api.openai.com/v1/responses")
  # If store=true: continue with previous_response_id
  # If store=false: compact + restart chain
```

Part 7: Developer Feedback and Competitive Landscape

What Developers Love About Claude Code

From community feedback across Reddit, benchmarks, and developer commentary:

- **Terminal UX:** "It just felt the most productive and easy to use; the CLI output design was easy to read and the task planning and use approval flows just seemed to work" (Render Benchmark)
- **Cost efficiency:** Claude Code offered the best cost efficiency at $8.67 for 29 minutes of use compared to competitors (Reddit)
- **Context management:** The compaction system, while complex, gives Claude Code ability to maintain coherent sessions that competitors lose track of
- **Permission flow:** The ask/allow/deny system is the right balance of safety and flow

What Developers Dislike

- **HTTP overhead:** Every turn re-sends the full conversation — wasteful on long sessions
- **Anthropic lock-in:** Transport layer is hardwired to Anthropic's API format; multi-provider requires proxy layers
- **Compaction artifacts:** Information loss during compaction can cause the agent to "forget" important context
- **No WebSocket:** No native support for persistent connections, which matters for the 20+ tool-call workflows that are Claude Code's primary use case

Codex CLI Feedback

- **Model quality:** "Codex performed admirably" with strong model reasoning, but "the UX felt somewhat primitive" (Render Benchmark)
- **Codex insistence:** When told output was broken, "it insisted I was mistaken and that the code was indeed faster" (Reddit)
- **App Server innovation:** The JSON-RPC bidirectional protocol is genuinely novel as a stable integration surface
- **Open source:** Full Rust codebase at github.com/openai/codex provides transparency

The Gap Both Miss

Neither Claude Code nor Codex CLI currently offers a single-binary, zero-proxy, WebSocket-native agentic coding tool that speaks the Responses API directly over persistent connections while maintaining the sophisticated harness architecture (permissions, hooks, multi-agent, memory) that makes Claude Code dominant. That gap is exactly what the proposed architecture fills.

Part 8: Feasibility Assessment

Can It Be Done? — Yes, With Zero Compromises

Every component of Claude Code's harness architecture is transport-agnostic:

Component	Transport Dependency	WebSocket-Native Path
Agentic loop	None	Identical
StreamingToolExecutor	Needs streaming events	WebSocket events are identical to SSE events
Tool system (80+ tools)	None	Identical
Permission system (3-tier)	None	Identical
Multi-agent mailbox	None	Multiple WebSocket connections for parallel subagents
Self-healing / circuit breakers	None	Identical
Hook system	None	Identical
Memory (AGENTS.md + auto)	None	Identical
Session persistence	None	Identical
MCP integration	Already supports WebSocket	Native WebSocket MCP when SEP-1287 ships
Compaction	HTTP-coupled	Replaced by server-side `context_management`
API call	HTTP-coupled	Replaced by `wss://` + `response.create`

Architectural Advantages of the WebSocket-Native Version

1. **~40% faster end-to-end** on 20+ tool-call workflows (OpenAI's own benchmark)
2. **Eliminates ~5,000+ lines** of client-side compaction code
3. **Zero context re-serialization** per turn — only incremental input
4. **Server-side compaction** handles the hard problem automatically
5. **Warmup requests** pre-load tools and instructions for faster first token
6. **Native multi-model routing** via `openai_prefix_mode` in the Agents SDK
7. **No proxy, no bridge, no HTTP** in the critical path
8. **ZDR-compatible** with `store=false` and connection-local memory
9. **MCP over WebSocket** possible when SEP-1287 ships — fully WebSocket-native stack

What the Architecture Cannot Do (Honest Limitations)

- **60-minute connection limit** requires reconnection handling with compact-and-restart
- **No multiplexing** — parallel subagents need separate WebSocket connections
- **Store=false + cache eviction** means `previous_response_not_found` on reconnect; must compact first
- **Server-side compaction is opaque** — the encrypted compaction item is not human-readable, so the client loses visibility into what was preserved
- **Provider lock-in shifts** from Anthropic to OpenAI (though `openai_prefix_mode` + OpenAI-compatible endpoints partially mitigate)
- **The Tier 2 ML classifier** in Claude Code's permission system calls the Claude API; a WebSocket-native version using OpenAI would need to reimplement this or use a model-agnostic classifier
- **Claude Code's custom React terminal renderer** (from-scratch Yoga flexbox port) is Anthropic-specific UI code that would need separate implementation

Part 9: Your Claw Code Fork — Where This Fits

Based on the repos found on your GitHub:

- claw-code — Your primary fork, described as "Better Harness Tools, now rewriting in Rust using oh-my-codex"
- claw-code-parity — Rust port parity work
- learn-claude-code — "Bash is all you need — A nano Claude Code-like agent harness, built from 0 to 1"
- **Multiple Claude Code source forks** — For reference study

Your PLAN-Claw-Code.md already maps the right architectural seams:

- `ConversationRuntime` in `conversation.rs` is the best existing seam — it already depends on a generic `ApiClient` trait and provider-neutral `AssistantEvents`
- The plan to introduce provider-neutral request/event/response types with `AnthropicClient` behind a concrete adapter is exactly the right foundation
- The proposed `ProviderFactory/ProviderRegistry` abstraction would naturally accommodate a WebSocket-native `OpenAIWebSocketProvider` alongside the existing `AnthropicHTTPProvider`

The Rust codebase's generic `ApiClient` trait is the perfect insertion point for a WebSocket transport implementation. The trait boundary already separates harness logic from wire protocol — the WebSocket provider just needs to implement the same interface using `wss://` frames instead of HTTP POST.

Conclusion

Claude Code's power comes from treating AI as a structured system — not a single model, but a governed execution environment with tools, permissions, agents, memory, hooks, and runtime all working together. That system architecture is completely independent of HTTP.

The WebSocket-native version retains every architectural advantage that makes Claude Code dominant — the agentic loop, streaming tool execution, permission gating, multi-agent orchestration, self-healing, and hooks — while eliminating the entire compaction tax that exists solely because HTTP is stateless. The Responses API WebSocket mode provides the missing transport layer, and the OpenAI Agents SDK proves the pattern works in production.

The honest answer to "can it be done?": not only is it possible, but the WebSocket-native version is strictly architecturally superior on every axis that mattered enough to Claude Code's engineers that they built an entire 5-stage compaction subsystem around it. The compaction problem is the HTTP problem. Remove HTTP, and the problem largely disappears.

Part II: Implementation Best Practices — Component-by-Component Official Guide

This addendum covers the official implementation patterns for every component in the WebSocket-native harness, drawn directly from OpenAI's documentation and the Agents SDK reference. Each section maps the official pattern to its role in the harness architecture, with specific guidance on compatibility and optimization when components are combined.

Part 10: Responses API Input Format and State Management

The Responses API replaces the Chat Completions message array with a richer input model built around typed Items. Understanding this format is foundational because every other component — tool calling, compaction, WebSocket mode — operates on these Items (OpenAI Migration Guide).

Input Structure

The Responses API accepts three input forms:

Input Type	When to Use	Harness Mapping
Plain string	Simple single-turn queries	Not applicable for agentic loop
Message list	`[{"role": "user", "content": "..."}]`	Initial user prompt on session start
Items list	Union of `message`, `function_call`, `function_call_output`, `reasoning`, `compaction`	Every continuation turn in the agentic loop

The Items format is what matters for the harness. Unlike Chat Completions where everything is a "message," the Responses API treats tool calls and tool outputs as first-class Item types with

their own schemas. A `function_call` Item has `call_id` and `name` fields; a `function_call_output` Item correlates back via the same `call_id` (OpenAI Function Calling Guide).

State Chaining with `previous_response_id`

The most architecturally significant feature for the harness is `previous_response_id`. It replaces Claude Code's entire "re-send the full conversation every turn" pattern:

- **First turn**: Send full `input` (system instructions + user message + tool definitions). Receive `response.id`.
- **Continuation turns**: Send `previous_response_id` + only new items (tool outputs, next user message). The server hydrates the full context from its stored state (OpenAI Conversation State Guide).
- **Over WebSocket**: The connection keeps the most recent response in a connection-local in-memory cache, so continuations don't even require disk hydration — they're served from memory, yielding up to 40% latency reduction for 20+ tool call workflows (OpenAI WebSocket Mode Guide).

This is the single biggest architectural win. Claude Code's 5-stage compaction pipeline exists because every HTTP POST must re-send the full conversation. With `previous_response_id` over WebSocket, the client sends only incremental additions.

The `store` Parameter and ZDR Compliance

A critical design decision for the harness:

Setting	Behavior	Use Case
`store: true` (default)	Server persists response for later retrieval	Persistent sessions, survives reconnection
`store: false`	No server-side persistence	ZDR compliance, ephemeral coding sessions
`store: false` + WebSocket	Connection-local cache only; lost on disconnect	Best latency + ZDR, but fragile to drops

Codex CLI deliberately avoids `previous_response_id` entirely for ZDR compliance (ZenML Codex Analysis). The WebSocket-native harness can have it both ways: use `store: false` for ZDR compliance while still benefiting from connection-local caching within a session. The tradeoff is that on reconnection with `store: false`, the `previous_response_id` is gone, so the harness needs a fallback to full context re-send or a local compacted snapshot.

Encrypted Reasoning for Stateless Workflows

For workflows that need ZDR but also want reasoning continuity: set `store: false` and include `"reasoning.encrypted_content"` in the `include` parameter. The server returns encrypted reasoning tokens that can be passed back in subsequent requests — the server decrypts them in-memory without persisting to disk (OpenAI Migration Guide).

Part 11: WebSocket Connection Management

The WebSocket transport is the harness's core transport layer. Every design decision flows from its characteristics (OpenAI WebSocket Mode Guide).

Connection Lifecycle

```
┌──────────────────────────────────────────────────────────┐
│  Client connects: wss://api.openai.com/v1/responses     │
│  Auth: Authorization: Bearer {API_KEY}                  │
├──────────────────────────────────────────────────────────┤
│  60-minute connection limit                             │
│  Sequential: one in-flight response.create at a time     │
│  No multiplexing per connection                         │
│  Connection-local cache: most recent response state      │
├──────────────────────────────────────────────────────────┤
│  On limit/close → reconnect + re-establish chain         │
└──────────────────────────────────────────────────────────┘
```

Key constraints the harness must design around:

- **60-minute hard limit**: The harness needs a reconnection manager that transparently re-establishes the WebSocket before the limit. Since agentic coding sessions routinely exceed 60 minutes, this is not optional.
- **Sequential execution**: Only one `response.create` can be in-flight per connection. For parallel tool execution (Claude Code runs reads in parallel), the harness either serializes the result submission or opens multiple connections.
- **No multiplexing**: If the harness needs to run multiple agents simultaneously (sub-agents, guardrails), each needs its own connection or they must be serialized.

Message Protocol

The client sends `response.create` events (not HTTP POST bodies). The payload is identical to the Responses API create body, except `stream` and `background` fields are omitted (streaming is implicit over WebSocket).

Initial turn:

```
{
  "type": "response.create",
  "model": "gpt-5.4",
  "store": false,
  "input": [{"type": "message", "role": "user", "content": [...]}],
  "tools": [...],
  "instructions": "...",
  "context_management": [{"type": "compaction", "compact_threshold": 200000}]
}
```

Continuation (after tool execution):

```
{
  "type": "response.create",
  "model": "gpt-5.4",
  "store": false,
  "previous_response_id": "resp_abc123",
  "input": [
    {"type": "function_call_output", "call_id": "call_xyz", "output": "..."},
    {"type": "message", "role": "user", "content": [{"type": "input_text", "text": "Co
  ]
}
```

Reconnection Strategy

The harness needs a three-tier reconnection strategy:

Scenario	Recovery Path
60-minute limit reached	Pre-emptive reconnect at ~55 min; continue with `previous_response_id` if `store: true`, or full context re-send if `store: false`
Network drop mid-response	Reconnect; if `store: true`, the in-flight response may have completed server-side — query its status. If `store: false`, re-send from last known good state
`previous_response_not_found` error	Server evicted the cached response. Fall back to full context re-send using locally maintained conversation state

The `previous_response_not_found` error (HTTP 400 equivalent over WebSocket) is the critical edge case. When the server returns this, the connection-local cache has been evicted (turn failure causes eviction). The harness must maintain a local shadow of the conversation state as a fallback — this is where a lightweight version of Claude Code's compaction pipeline still adds value, not as the primary path but as the recovery path.

Warmup Requests

WebSocket mode supports a `generate: false` warmup request that prepares server state without generating a response. This returns a `response.id` that can be used as `previous_response_id` for the actual generation. Use this to pre-load the system prompt and tool definitions at session start, so the first real user turn only sends the user message as incremental input.

Part 12: Function Calling — Strict Mode, Namespaces, and Tool Search

Claude Code defines 25+ tools (Read, Write, Bash, Grep, Glob, etc.). The Responses API's function calling system has several features that directly map to how the harness should expose these tools (OpenAI Function Calling Guide).

Strict Mode (Default in Responses API)

Unlike Chat Completions where `strict` was opt-in, the Responses API defaults to strict function calling. This means:

- The model's generated function arguments are guaranteed to conform to the JSON Schema provided.
- The API validates arguments server-side and will not return malformed calls.
- The harness can skip client-side schema validation of tool arguments entirely — the server guarantees conformance.

This eliminates one of Claude Code's failure modes where the model generates malformed tool calls that need to be caught and re-prompted.

Tool Namespaces

For a harness with 25+ tools, namespacing prevents tool name collisions and improves model tool selection:

```
from agents import Agent, function_tool, tool_namespace

@tool_namespace("filesystem")
@function_tool
def read_file(path: str) -> str: ...

@tool_namespace("filesystem")
@function_tool
def write_file(path: str, content: str) -> str: ...

@tool_namespace("search")
@function_tool
def grep(pattern: str, path: str) -> str: ...
```

The model sees these as `filesystem.read_file`, `filesystem.write_file`, `search.grep` — clearer signal about tool categories.

Tool Search (Deferred Loading)

For large tool sets, `ToolSearchTool` (available on `gpt-5.4` and later) lets the model dynamically load tools instead of receiving all definitions upfront (OpenAI Function Calling Guide):

```
from agents import Agent, ToolSearchTool

agent = Agent(
    name="Coder",
    tools=[ToolSearchTool(), ...core_tools...],
)
```

The model loads rarely-used tools on demand. For the harness:

- **Always-loaded**: Read, Write, Bash, Grep (used every turn)
- **Deferred**: Notebook tools, image tools, specialized formatters
- **Benefit**: Reduces prompt token count per turn, improving prompt cache hit rates

Parallel Tool Calls

The Responses API supports parallel tool calls by default. Claude Code's pattern of "reads in parallel, writes serial" maps directly:

```
agent = Agent(
    name="Coder",
    model_settings=ModelSettings(
        parallel_tool_calls=True,  # default
    ),
)
```

When the model returns multiple `function_call` items in one response, execute reads concurrently and writes sequentially (the harness's permission system gates writes anyway). Feed all `function_call_output` items back in one `response.create` continuation.

Tool Choice Control

Setting	Behavior	Harness Use
`tool_choice: "auto"`	Model decides (default)	Normal agentic loop turns
`tool_choice: "required"`	Must call at least one tool	Force tool use when model stalls
`tool_choice: {"type": "function", "name": "bash"}`	Force specific tool	Recovery/self-healing
`tool_choice: "none"`	No tool calls allowed	Final summary generation

Claude Code's "self-healing" pattern (re-prompting with `tool_choice: required` after a stall) maps directly to the Responses API's tool choice override.

Part 13: Streaming Event Handling — The 53-Event Taxonomy

The Responses API defines 53 distinct streaming event types. The harness must handle all of them correctly for a production-quality terminal experience (Streaming Events Guide).

Event Categories

Category	Events	Harness Role
Response Envelope	`response.created`, `.in_progress`, `.completed`, `.incomplete`, `.failed`, `error`	Session state machine

Category	Events	Harness Role
Text Output	`output_item.added`, `content_part.added`, `output_text.delta`, `output_text.done`	Terminal streaming renderer
Function Calls	`output_item.added` (type=function_call), `function_call_arguments.delta`, `function_call_arguments.done`	Tool dispatch
Reasoning	`reasoning_summary_part.added`, `reasoning_summary_text.delta/done`	Optional reasoning display
MCP Calls	`mcp_call_arguments.delta/done`, `mcp_call.in_progress/completed/failed`	MCP tool integration
Built-in Tools	`web_search_call.*`, `file_search_call.*`, `code_interpreter_call.*`	Native tool status display

Critical Implementation Rules

1. **Route by `event:` line, parse `data:` as JSON.** Never assume event ordering beyond the documented lifecycle.
2. **Maintain three index maps:**
   - `response.id` → global response state
   - `(output_index, item_id)` → per-item buffer
   - `(item_id, content_index)` → per-content-part buffer
3. **Append `.delta` text in `sequence_number` order.** Only trust `.done` text as ground truth — if the `.done` text differs from accumulated deltas (rare but possible), replace.
4. **Only read `usage` from `response.completed`.** All prior response echoes contain `usage: null`.
5. **On `response.incomplete` or `response.failed`, stop assembling.** Present the reason/error and decide whether to retry.
6. **Ignore unknown keys** for forward compatibility.

The Function Call Lifecycle

This is the most important lifecycle for the agentic loop:

```
response.output_item.added (type=function_call, name="bash", call_id="call_abc")
  → function_call_arguments.delta × N  (accumulate JSON string)
  → function_call_arguments.done       (parse JSON, dispatch tool)
  → response.output_item.done          (tool call item complete)
```

The harness should:

1. On `output_item.added` with `type=function_call`: Create a pending tool call entry, display tool name in terminal.

2. On `function_call_arguments.delta`: Accumulate raw JSON string (do not parse until done).
3. On `function_call_arguments.done`: Parse JSON arguments, run through permission gate, execute tool.
4. Collect all tool outputs, then send a single `response.create` continuation with all `function_call_output` items.

Multiple Tool Calls in One Response

When the model returns multiple function calls, they appear as separate `output_item.added` events with different `output_index` values. The harness should:

- Collect all tool calls until `response.completed` (or until all tool `output_item.done` events received)
- Execute reads in parallel, writes in sequence
- Submit all `function_call_output` items together in one continuation

Part 14: Server-Side Compaction — Replacing the 5-Stage Pipeline

Claude Code's compaction is a 5-tier client-side system. The Responses API provides server-side compaction that replaces most of it with a single configuration parameter (OpenAI Compaction Guide).

Enabling Compaction

```
{
  "context_management": [{"type": "compaction", "compact_threshold": 200000}]
}
```

Set this on every `response.create` call. When the rendered token count crosses `compact_threshold`, the server:

1. Triggers a compaction pass during inference
2. Emits an encrypted compaction item in the response stream
3. Prunes context before continuing

How It Interacts with `previous_response_id`

Two chaining modes after compaction:

Mode	What to Do	When to Use
`previous_response_id` chaining	Pass only new items; server handles pruning internally	Default mode — simplest
Stateless input-array chaining	Append all output items (including compaction items) to input; drop items before most recent compaction item	ZDR / `store: false` / recovery

With `previous_response_id` chaining over WebSocket, the client literally does nothing special for compaction — the server handles everything. This is the core of "the HTTP tax disappears."

Client-Side Compaction as Fallback

The harness should still maintain a lightweight client-side compaction fallback for:

- Recovery from `previous_response_not_found` (reconnection scenarios)
- Sessions using `store: false` that drop the WebSocket connection
- Explicit `/responses/compact` endpoint calls for manual context windowing

The standalone `/responses/compact` endpoint accepts a conversation and returns a compacted version. Use this when the harness needs to rebuild context after a connection loss.

Compaction Threshold Selection

Model	Max Context	Recommended Threshold
gpt-5.4	1M tokens	200,000 (comfortable margin)
gpt-5.1-codex	1M tokens	200,000
gpt-4.1	1M tokens	200,000

Set the threshold well below max context to leave headroom for the model's output generation. The Codex harness uses 200,000 as its default.

Part 15: Prompt Caching — Maximizing the 90% Cost Reduction

Prompt caching reduces input token costs by up to 90% and latency by up to 80%. For an agentic coding tool making dozens of API calls per session, this is a massive optimization (OpenAI Prompt Caching Guide).

How It Works

1. Requests are routed to a machine based on a hash of the initial prefix (~256 tokens).
2. If the full prefix (1024+ tokens) matches a cached entry on that machine, it's a cache hit.
3. Cached prefixes remain active for 5-10 minutes (in-memory) or up to 24 hours (extended retention).

Harness Optimization Strategy

The key insight: **put static content at the beginning of every request, dynamic content at the end.**

For the agentic coding harness, the optimal prompt structure is:

```
┌──────────────────────────────────────────┐  ← STATIC PREFIX (cacheable)
│  System instructions                     │
```

```
│  Tool definitions                          │
│  CLAUDE.md / AGENTS.md equivalent           │
│  Permission rules                          │
│  Project context (file tree, conventions)   │
├──────────────────────────────────────────────┤  ← DYNAMIC SUFFIX (per-turn)
│  Conversation history / previous_response_id│
│  Current user message                      │
│  Tool outputs from this turn               │
└──────────────────────────────────────────────┘
```

`prompt_cache_key` for Session Affinity

The `prompt_cache_key` parameter influences routing. For the harness:

```
{
  "prompt_cache_key": "session_abc123",
  "prompt_cache_retention": "24h"
}
```

- Use the session ID as `prompt_cache_key` so all requests in a coding session route to the same cache.
- Keep each unique prefix-key combination below ~15 requests/minute to avoid cache overflow.
- Use `"24h"` extended retention for long coding sessions (available on gpt-5.4, gpt-5.1-codex, gpt-4.1).

Extended vs In-Memory Retention

Retention	Duration	ZDR Compatible	Available Models
`in_memory` (default)	5-10 min active, max 1 hour	Yes	All models supporting caching
24h (extended)	Up to 24 hours	No (KV tensors stored to GPU-local storage)	gpt-5.4, gpt-5.1-codex, gpt-5, gpt-4.1 and variants

For ZDR-compliant workflows, use `in_memory` retention. For maximum performance in non-ZDR sessions, use `24h`.

Compatibility with WebSocket Mode

Prompt caching works identically over WebSocket and HTTP — it operates at the inference layer, not the transport layer. However, the combination of WebSocket connection-local caching + prompt caching is multiplicatively beneficial:

- **WebSocket cache**: Eliminates re-sending the full conversation (transport-level)
- **Prompt cache**: Eliminates re-computing attention for the static prefix (compute-level)
- **Together**: The harness sends minimal incremental data over the wire AND the server skips attention computation for the cached prefix

Part 16: Agents SDK — Runner, Handoffs, Guardrails, and Tracing

The OpenAI Agents SDK provides production-grade implementations of the exact patterns the harness needs. While the harness may not use the SDK directly (it's building a custom Rust runtime), the SDK's patterns are the canonical reference for how these components should work together (Agents SDK Overview).

The Runner Loop

The SDK's `Runner.run()` implements the same while-loop architecture as Claude Code's `query.ts` (Agents SDK Running Agents):

1. Call LLM for current agent with current input.
2. If LLM returns `final_output` (text output, no tool calls) → loop ends.
3. If LLM does a handoff → update current agent and input, re-run loop.
4. If LLM produces tool calls → execute tools, append results, re-run loop.
5. If `max_turns` exceeded → raise `MaxTurnsExceeded` (overridable via error handlers).

The harness should implement this same structure, with the addition of Claude Code's permission gating between steps 2 and 3.

WebSocket Session Pattern

The SDK's `responses_websocket_session()` is the production proof of the WebSocket-native pattern (Agents SDK WebSocket Session Reference):

```
async with responses_websocket_session() as ws:
    # First turn
    first = ws.run_streamed(agent, "Read the file src/main.rs")
    async for event in first.stream_events():
        handle(event)

    # Second turn — previous_response_id chaining
    second = ws.run_streamed(
        agent,
        "Now refactor the error handling",
        previous_response_id=first.last_response_id
    )
    async for event in second.stream_events():
        handle(event)
```

The session creates a shared `MultiProvider` with `openai_use_responses_websocket=True`, maintains connection pooling, and enables `previous_response_id` chaining across runs.

Handoffs for Multi-Agent Orchestration

Claude Code's sub-agent system maps to the Agents SDK's handoff pattern (Agents SDK Handoffs):

Claude Code Pattern	Agents SDK Equivalent
Main agent spawns sub-agent	`Agent(handoffs=[sub_agent])`
Sub-agent has restricted tools	`handoff(sub_agent, input_filter=...)`
Sub-agent returns results to parent	Handoff stays within single run
Sub-agent permission inheritance	`on_handoff` callback for permission propagation

Key implementation details:

- Handoffs appear as tools to the LLM (e.g., `transfer_to_refactor_agent`)
- `input_filter` controls what conversation history the sub-agent sees (critical for context management)
- `on_handoff` callback can propagate permission state, budget tracking, and session context
- `nest_handoff_history` (opt-in) collapses prior transcript into a summary before handing off — useful for keeping sub-agent context small

Guardrails for Permission Gating

Claude Code's permission system (deny → allow → ask user) maps to the Agents SDK's guardrail architecture (Agents SDK Guardrails):

Guardrail Type	Execution	Harness Use
Input guardrails	Run on initial user input	Content policy, prompt injection detection
Output guardrails	Run on final agent output	Sensitive data filtering
Tool guardrails (input)	Before each tool execution	Permission gating (the core Claude Code pattern)
Tool guardrails (output)	After each tool execution	Output sanitization

The tool guardrail pattern is the most important for the harness:

```
@tool_input_guardrail
def permission_gate(data):
    args = json.loads(data.context.tool_arguments or "{}")
    tool = data.context.tool_name

    # Claude Code's deny → allow → ask pattern
    if is_denied(tool, args):
        return ToolGuardrailFunctionOutput.reject_content("Operation denied by policy.")
    if needs_approval(tool, args):
        approved = await ask_user_permission(tool, args)
        if not approved:
```

```
            return ToolGuardrailFunctionOutput.reject_content("User rejected.")
    return ToolGuardrailFunctionOutput.allow()
```

Input guardrails can run in parallel with the model (`run_in_parallel=True`, default) for best latency, or blocking (`run_in_parallel=False`) to prevent token consumption on rejected inputs.

Context Management (RunContextWrapper)

The SDK's `RunContextWrapper` provides dependency injection across agents, tools, and callbacks (Agents SDK Context):

```
@dataclass
class HarnessContext:
    session_id: str
    project_root: Path
    permission_policy: PermissionPolicy
    budget_tracker: BudgetTracker
    git_state: GitState
    logger: Logger

agent = Agent[HarnessContext](
    name="Coder",
    tools=[read_file, write_file, bash],
)

result = await Runner.run(agent, input, context=harness_ctx)
```

The context object is purely local — never sent to the LLM. Tools access it via `wrapper.context`. This is how the harness shares session state, permission policies, and budget tracking across the entire agent graph.

Tracing for Observability

The Agents SDK provides automatic tracing with 25+ external integration targets (Agents SDK Tracing):

- Every `Runner.run()` creates a trace
- Agent runs, LLM generations, tool calls, guardrails, and handoffs each create spans
- Traces are sent to OpenAI's backend by default (free dashboard)
- Custom `TraceProcessor` implementations can route to Langfuse, Braintrust, PostHog, or any other backend
- `trace_include_sensitive_data=False` redacts LLM inputs/outputs from spans
- Under ZDR policies, tracing to OpenAI's backend is unavailable

For the harness, tracing maps to Claude Code's telemetry system — but with far more integration options and a standardized span model.

Model Configuration

The SDK's `ModelSettings` exposes Responses API-specific parameters that are critical for the harness (Agents SDK Models):

```
ModelSettings(
    parallel_tool_calls=True,          # Enable parallel reads
    truncation="auto",                 # Drop oldest items on overflow
    store=False,                       # ZDR compliance
    prompt_cache_retention="24h",        # Extended caching
    response_include=[                 # Rich payloads
        "reasoning.encrypted_content",
    ],
    retry=ModelRetrySettings(          # Runner-managed retries
        max_retries=4,
        backoff={"initial_delay": 0.5, "max_delay": 5.0, "multiplier": 2.0, "jitter": Tr
        policy=retry_policies.any(
            retry_policies.provider_suggested(),
            retry_policies.retry_after(),
            retry_policies.network_error(),
            retry_policies.http_status([408, 429, 500, 502, 503, 504]),
        ),
    ),
)
```

The `retry` configuration is especially important: stateful requests using `previous_response_id` need `provider_suggested()` policy because the server knows whether a retry is safe for stateful continuations.

Part 17: MCP Tool Integration

The Model Context Protocol defines how the harness connects to external tool servers. MCP is transport-agnostic, making it naturally compatible with the WebSocket-native architecture (MCP Transports Specification).

Standard Transports

Transport	Mechanism	Harness Use
stdio	Client launches MCP server as subprocess; JSON-RPC over stdin/stdout	Local tools (LSP, linters, formatters)
Streamable HTTP	HTTP POST/GET with optional SSE streaming	Remote tools (hosted services)
Custom	Any bidirectional channel supporting JSON-RPC	WebSocket-native MCP (future)

The MCP spec explicitly states: "Clients and servers MAY implement additional custom transport mechanisms… The protocol is transport-agnostic and can be implemented over any communication channel that supports bidirectional message exchange." This means a WebSocket transport for MCP is spec-compliant.

Current Status: No Native WebSocket Transport

As of the 2025-11-25 MCP specification, there is no standard WebSocket transport. The old HTTP+SSE transport was deprecated in favor of Streamable HTTP. A WebSocket transport proposal exists as SEP-1287 but is not yet adopted.

Integration Architecture

The harness connects to MCP servers through the standard transports while itself using WebSocket for the LLM connection:

```
┌──────────────────────────────────────────────────┐
│  WebSocket-Native Harness                        │
│                                                 │
│  ┌──────────────────────────────────────────┐   │
│  │  MCP Client                              │   │
│  │  ├─ stdio → local LSP server             │   │
│  │  ├─ stdio → local linter                 │   │
│  │  └─ Streamable HTTP → remote code search │   │
│  └──────────────────────────────────────────┘   │
│                   ↕ tool results                 │
│  ┌──────────────────────────────────────────┐   │
│  │  Agentic Loop                            │   │
│  │  ├─ WebSocket → OpenAI Responses API     │   │
│  │  ├─ Tool execution engine                │   │
│  │  └─ Permission gate                      │   │
│  └──────────────────────────────────────────┘   │
└──────────────────────────────────────────────────┘
```

MCP in the Responses API

The Responses API has native MCP support. The model can call MCP tools directly, with the server handling the MCP protocol:

- `mcp_call` output items appear in the streaming event stream
- `mcp_call_arguments.delta/done` events stream the tool arguments
- `mcp_call.in_progress/completed/failed` events track execution status

This means the harness can expose local MCP servers to the model via the Responses API's built-in MCP tool type, rather than implementing MCP→function_call bridging manually.

Session Management

MCP sessions use `MCP-Session-Id` headers for stateful interactions. The harness's MCP client should:

1. Initialize each MCP server connection with `InitializeRequest`
2. Store the returned `MCP-Session-Id`
3. Include it in all subsequent requests

4. Handle `404 Not Found` (session expired) by re-initializing
5. Send `DELETE` on session teardown

Part 18: Harness Engineering Patterns from Codex

OpenAI's harness engineering approach — documented in their blog post on building an entire product using only Codex agents — provides patterns directly applicable to the WebSocket-native harness (OpenAI Harness Engineering).

Agent-Legible Environment Design

The Codex team's core insight: the quality of agent output is bounded by the quality of the environment the agent operates in. Their key patterns:

- **Progressive disclosure**: Small entry points (like `AGENTS.md`) that point to deeper documentation. The harness's system prompt should follow this: short top-level instructions with `@file` references to detailed docs.
- **Repository-local knowledge**: All knowledge lives in the repo (code, markdown, schemas, plans). No external sources the agent can't access. The harness should inject `AGENTS.md` + `docs/` tree into the system prompt prefix.
- **Mechanically enforced invariants**: Linters, structural tests, and CI validations that agents can run and learn from. Linter error messages include remediation instructions — the agent reads the error and knows how to fix it.

Tool Autonomy Pattern

Codex agents use standard tools directly:

- `gh` (GitHub CLI) for PR operations
- Chrome DevTools Protocol for UI validation
- Local observability stack (logs, metrics, traces) exposed as agent-readable

The harness should follow this: tools should be real system tools (not wrappers), and their output should be structured for LLM consumption.

The Self-Review Loop

Codex's "Ralph Wiggum Loop":

1. Agent implements changes
2. Agent reviews its own changes locally
3. Agent requests review from other agents
4. Agent responds to feedback and iterates
5. All reviewers satisfied → merge

This maps to the harness's multi-agent capability: a coding agent hands off to a review agent, which can hand off to a test agent, all within a single `Runner.run()` via the handoff pattern.

Context Management via Documentation

Codex maintains ~100-line `AGENTS.md` as a table of contents pointing to `docs/` directory:

Document Type	Purpose
Design docs	Catalogued with verification status
Architecture docs	Top-level domain/package map
Quality doc	Grades product domains, tracks gaps
Plans	Ephemeral (small changes) or execution plans (complex work)

The harness should support an equivalent: the system prompt includes a project map, and the model can use `Read` tools to load deeper documentation on demand.

Part 19: Putting It All Together — The Optimized Request Lifecycle

Here is the complete lifecycle of a single turn in the WebSocket-native harness, incorporating all the best practices from Parts 10-18:

Session Initialization

```
1. Open WebSocket: wss://api.openai.com/v1/responses
   Auth: Bearer {API_KEY}

2. Warmup request (generate: false):
   {
     "type": "response.create",
     "generate": false,
     "model": "gpt-5.4",
     "store": false,
     "instructions": "<system prompt + AGENTS.md>",
     "tools": [<core tools>],
     "tool_search": {<deferred tools>},
     "prompt_cache_key": "session_abc",
     "prompt_cache_retention": "24h",
     "context_management": [{"type": "compaction", "compact_threshold": 200000}]
   }
   → Receive response.id (warmup_id)
   → System prompt + tools now cached server-side

3. Initialize MCP servers (stdio for local, Streamable HTTP for remote)
```

Each Agentic Loop Turn

```
1. SEND response.create:
   {
     "type": "response.create",
     "previous_response_id": "<last_response_id>",
     "input": [<only new items: user message or tool outputs>]
   }

2. RECEIVE streaming events:
   - response.created → stash response.id
   - response.in_progress → update terminal status
   - For each output_item.added:
     - type=reasoning → display thinking indicator
     - type=message → stream text to terminal via output_text.delta
     - type=function_call → accumulate arguments via function_call_arguments.delta
   - function_call_arguments.done → PERMISSION GATE:
     - Check deny list → reject
     - Check allow list → execute
     - Otherwise → prompt user
   - response.completed → read usage, check for more tool calls

3. IF tool calls present:
   - Execute reads in parallel, writes in sequence
   - Collect all function_call_output items
   - GOTO step 1 with outputs as new input

4. IF no tool calls (final_output):
   - Display response
   - Wait for next user input
   - GOTO step 1 with user message as new input

5. IF compaction triggered (compaction item in stream):
   - Server handles automatically
   - If using previous_response_id: no client action needed
   - If using input-array chaining: drop items before compaction item

6. IF error/disconnect:
   - Reconnect WebSocket
   - If store=true: continue with previous_response_id
   - If store=false: fall back to local context snapshot
```

Cost and Latency Profile

Metric	Claude Code (HTTP)	WebSocket-Native Harness	Improvement
Per-turn transport overhead	Full context re-send	Incremental items only	50-90% reduction
Context computation	Full attention every turn	Prompt cache hit on static prefix	Up to 90% cost reduction
Tool call latency (20+ calls)	20× HTTP round-trips	Single persistent connection	~40% latency reduction

Metric	Claude Code (HTTP)	WebSocket-Native Harness	Improvement
Compaction complexity	5-tier client-side pipeline	Single server parameter	Massive code reduction
Reconnection cost	N/A (stateless)	Graceful degradation with fallback	Equivalent worst-case

Conclusion — Part II

Every component in the OpenAI stack — Responses API input format, WebSocket transport, server-side compaction, prompt caching, function calling with strict mode, the Agents SDK's Runner/handoffs/guardrails/tracing patterns, and MCP integration — has been designed to work together. The official documentation makes this interoperability explicit: `responses_websocket_session()` combines the WebSocket transport with the Runner loop, `context_management` integrates compaction into the response stream, `prompt_cache_key` with `prompt_cache_retention` optimizes caching across the session, and tool guardrails implement Claude Code's permission pattern.

No single component is individually unprecedented. What is unprecedented is combining all of them with Claude Code's harness architecture into a single WebSocket-native runtime. The official docs confirm that every pairwise integration works — WebSocket + compaction, compaction + prompt caching, prompt caching + tool search, tool search + guardrails, guardrails + handoffs. The harness architecture described here is the natural synthesis: the union of Claude Code's proven agentic loop design with the full depth of OpenAI's Responses API capabilities, connected by a persistent WebSocket that eliminates the transport layer entirely as a source of complexity.